

Automatic Vietnamese Text Summarization Using a Custom Transformer and a Fine-tuned Pre-trained Language Model

*Course Project Report for Natural Language Processing (NLP)

Nguyen Viet Thanh Lan

VNU-UET

Student ID: 23020099

GitHub

: <https://github.com/thanhlan89/BTL-NLP>

Abstract

This paper addresses the task of abstractive Vietnamese text summarization through two parallel approaches required by the project specification: (1) building a sequence-to-sequence Transformer entirely from scratch, and (2) fine-tuning a pre-trained Vietnamese language model. For the first approach, we re-implement the base Transformer of “Attention Is All You Need” in PyTorch without any high-level transformer modules, and progressively add modern enhancements: Pre-Layer Normalization (Pre-LN), Rotary Position Embedding (RoPE), a SwiGLU feed-forward network, and a Pointer-Generator copy mechanism with a coverage penalty. For the second approach, we fine-tune VietAI/vit5-base (227M parameters, well under the 3B limit) using Parameter-Efficient Fine-Tuning with Low-Rank Adaptation (LoRA), and further explore reinforcement learning via Self-Critical Sequence Training (SCST) to directly optimize sequence-level rewards. We also describe a dedicated text-cleaning pipeline that repairs common Vietnamese crawling artifacts (glued tokens, control characters, leakage). Evaluated on a Vietnamese news corpus of 10,775 training and 1,349 validation pairs, the fine-tuned ViT5-base with LoRA yields the highest ROUGE-L score (ROUGE-L = 0.4767), while our custom Transformer reaches a competitive ROUGE-L = 0.4451 despite training from scratch on a limited dataset.

Index Terms—Text Summarization, Transformer from Scratch, ViT5, LoRA, SCST, Reinforcement Learning, Vietnamese NLP.

I. INTRODUCTION

Automatic text summarization is a core Natural Language Processing (NLP) task that compresses a long document

into a shorter form while preserving its essential meaning. Two paradigms exist: *extractive* summarization, which copies salient sentences verbatim, and *abstractive* summarization, which generates new sentences and therefore demands a deeper grasp of syntax and semantics. This project targets the harder abstractive setting for Vietnamese, a language with rich diacritics and flexible word order.

The motivation is practical: the volume of Vietnamese online news grows continuously, and readers increasingly rely on concise summaries to triage information. Abstractive systems, however, are prone to characteristic failure modes—word repetition, factual hallucination, and omission of late-document details—that make the task both useful and technically demanding.

Following the project specification, we study two approaches in parallel and give each equal weight. The first is a Transformer built entirely from scratch, which lets us control every architectural detail and quantify the effect of individual enhancements. The second fine-tunes a strong pre-trained Vietnamese model under a tight parameter budget. Our contributions are:

- A faithful from-scratch implementation of the base Transformer [1], extended with Pre-LN, RoPE, SwiGLU and a pointer-generator/coverage mechanism, implemented without any high-level transformer modules.
- A parameter-efficient fine-tuning recipe for ViT5 that combines LoRA with SCST reinforcement learning to directly optimize the ROUGE-L objective.
- A robust Vietnamese text-cleaning pipeline that repairs common crawling artifacts and prevents train/validation leakage.
- A controlled comparison of both approaches under standard metrics (ROUGE, BLEU, chrF, BERTScore), including an ablation-style comparison of successive architectural variants.

II. RELATED WORK

Early summarization systems were predominantly extractive, ranking sentences with graph-based methods such as TextRank or with statistical surface features. These methods guarantee grammaticality because they reuse original sentences, but they cannot paraphrase, merge information, or compress at the sub-sentence level. The introduction of the Seq2Seq encoder-decoder with recurrent networks and attention enabled genuinely abstractive generation, yet recurrence limited parallelism and the modeling of long-range dependencies.

The Transformer of Vaswani et al. [1] removed recurrence entirely, relying on self-attention to capture dependencies at any distance while enabling full parallelization across sequence positions. This architecture underpins large pre-trained models such as T5 [2], which casts every NLP task in a unified text-to-text format and therefore applies naturally to summarization. For Vietnamese, Phan et al. [3] released ViT5, a T5 model pre-trained on large Vietnamese corpora that reaches state-of-the-art results on generation tasks including abstractive summarization.

To adapt such models cheaply, Parameter-Efficient Fine-Tuning (PEFT) has become standard practice. LoRA [4] freezes the pre-trained weights and learns small low-rank update matrices, drastically cutting the number of trainable parameters and the memory footprint while limiting overfitting on small downstream datasets. To address the mismatch between token-level maximum-likelihood training and sequence-level evaluation, reinforcement-learning methods such as Self-Critical Sequence Training (SCST) [5] optimize the evaluation metric directly through a policy gradient.

On the architecture side, several enhancements have proven broadly effective. Pre-Layer Normalization improves gradient flow and training stability in deep Transformer stacks. RoPE [6] encodes relative positions by rotating queries and keys, generalizing better to varying sequence lengths than additive sinusoidal embeddings. GLU variants such as SwiGLU [7] increase the expressiveness of the feed-forward block. Finally, for faithfulness in summarization, the Pointer-Generator network with a coverage penalty [8] mitigates hallucination and repetition by copying source tokens and explicitly tracking the history of attention. Our work combines these ideas in a single from-scratch model and contrasts them with a PEFT-tuned pre-trained baseline.

III. METHODOLOGY

We develop two parallel architectures and evaluate them under an identical data pipeline and metric suite. Section III-A describes preprocessing; Section III-B the from-scratch base Transformer; Section III-C the architectural enhancements with a detailed justification of *why* each one helps; Section III-D the pointer-generator and coverage mechanism; and Section III-E the pre-trained ViT5 model with LoRA and SCST.

A. Data Preparation and Cleaning

Both models share one preprocessing pipeline. Raw Vietnamese news pairs contain frequent crawling artifacts, which, if left untreated, are memorized and reproduced by the decoder. We therefore apply, in order:

- 1) **Unicode NFC normalization**, collapsing combining diacritics into pre-composed Vietnamese characters so that identical surface forms map to identical token sequences.
- 2) **Noise removal**: zero-width characters, soft hyphens, control characters, residual HTML tags, URLs and email addresses are stripped.
- 3) **Glued-token repair**: tokens fused by a missing space are split at camel-case boundaries (`caGặp` → `ca Gặp`) and at digit-letter boundaries (`60c6` → `60 c6`, `ta60` → `ta 60`).

After normalization, pairs are filtered by length (source 20–1500 words, summary 5–200 words), by an admissible compression ratio ($1.2 \leq |\text{src}|/|\text{tgt}| \leq 60$), and by the absence of abnormally long glued runs (≥ 18 letters) that signal irreparably corrupted text. Exact duplicates are removed, and validation samples whose source also occurs in the training set are dropped to prevent leakage that would otherwise inflate ROUGE. This pipeline directly removed the copy-of-noise errors observed in early qualitative outputs.

B. Base Transformer Architecture (Scratch Implementation)

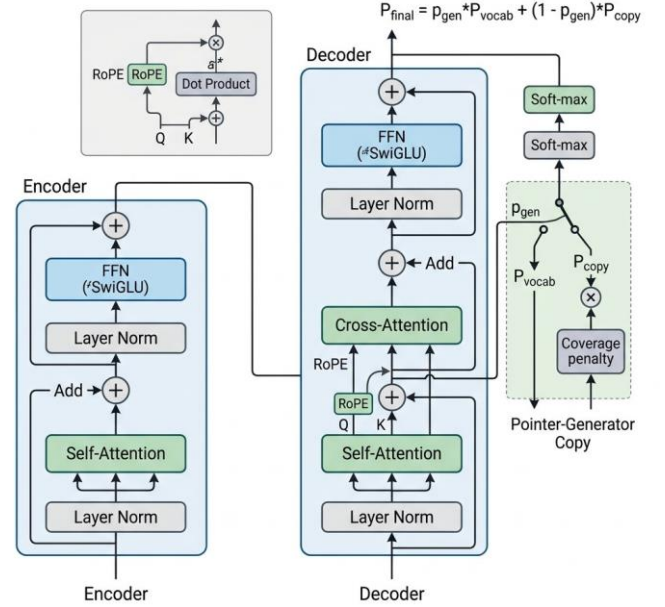


Figure 1. Block diagram of our custom Seq2Seq Transformer architecture, showing the integration of Pre-LN, RoPE, SwiGLU, and the Pointer-Generator copy/coverage mechanism.

We implemented the Encoder-Decoder Transformer from scratch in PyTorch, deliberately avoiding high-level abstractions such as

nn.Transformer, nn.MultiheadAttention or F.scaled_dot_product_attention; only primitive layers (nn.Linear, nn.Embedding, nn.LayerNorm, nn.Dropout) are used. The model follows the formulation of Vaswani et al. [1].

1) *Token and Positional Representation*: The input subword sequence $\mathbf{x}_i = (x_1, \dots, x_{T_x})$ is embedded into $\mathbb{R}^{d_{\text{model}}}$ and scaled by $\sqrt{d_{\text{model}}}$ so that embedding magnitudes match the positional signal. The encoder maps $\mathbf{x}$ to contextual states $\mathbf{h} = (h_1, \dots, h_{T_x})$; the decoder autoregressively produces target tokens $\mathbf{y} = (y_1, \dots, y_{T_y})$.

2) *Scaled Dot-Product Attention*: The central operation computes, for queries Q , keys K and values V ,

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\sqrt{\frac{QK}{d_k}} V \right), \quad (1)$$

where d_k is the per-head dimension. The factor $1/\sqrt{d_k}$ rescales the dot products: without it, for large d_k the products grow in magnitude and push the softmax into saturated regions with vanishing gradients.

3) *Multi-Head Attention*: Instead of a single attention over the full d_{model} space, h heads attend in parallel subspaces:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \quad (2)$$

with $\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$ and learned projections $W_i, W_i \in \mathbb{R}^{\text{model} \times k}$, $W_i \in \mathbb{R}^{d_{\text{model}} \times v}$, $W \in \mathbb{R}^{h d_v \times d_{\text{model}}}$. Multiple heads let the model attend to different relations (syntactic, positional, lexical) simultaneously.

4) *Masking*: Two masks are applied explicitly. A *padding mask* zeroes attention to padding positions in both stacks. A *causal mask* $M_{ij} = -\infty$ for $j > i$ in the decoder self-attention enforces autoregression so that position i cannot attend to future tokens.

5) *Feed-Forward, Residuals and Output*: Each sub-layer is wrapped with a residual connection and layer normalization, and every layer ends with a position-wise FFN. Output token probabilities are produced by a linear projection whose weight is tied to the input embedding matrix, reducing parameters and regularizing the output space.

C. Advanced Architectural Enhancements

Over the base model we integrate three enhancements. For each we give the formulation and a detailed argument for *why* it improves training or quality on our task.

1) *Pre-Layer Normalization (Pre-LN)*: The original Transformer normalizes *after* the residual addition (Post-LN):

$$\mathbf{x}_{i+1} = \text{LayerNorm}(\mathbf{x}_i + \text{SubLayer}(\mathbf{x}_i)). \quad (3)$$

We instead normalize *inside* the residual branch (Pre-LN):

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \text{SubLayer}(\text{LayerNorm}(\mathbf{x}_i)). \quad (4)$$

Why it works. In Post-LN, every residual signal passes through a normalization layer before being forwarded, so the backward gradient is repeatedly scaled by the LayerNorm Jacobian; in deep stacks this compounds and destabilizes early-layer gradients, forcing a long learning-rate warmup

and careful initialization. Pre-LN leaves a *clean identity path* $\mathbf{x}_i \rightarrow \mathbf{x}_{i+1}$ through which gradients flow undistorted to the earliest layers. The practical effects we observe (Table II) are faster, more stable convergence and a large jump in ROUGE-L over the Post-LN baseline, with far less sensitivity to the warmup schedule.

2) *Rotary Position Embedding (RoPE)*: Rather than adding static sinusoidal vectors to token embeddings, RoPE rotates the query and key vectors by a position-dependent angle [6]:

$$R_{\Theta, m}^d = \text{diag} \ R_{\Theta, m}, \dots, R_{\Theta, d/2, m}, \quad (5)$$

where $R_{\Theta, m}$ is a 2×2 rotation by angle $m\Theta$. The key property is that the post-rotation inner product depends only on the relative offset $m - n$:

$$\langle R_{\Theta, m}^d q_m, R_{\Theta, n}^d k_n \rangle = g(q_n, k_n, m - n). \quad (6)$$

Why it works. Additive sinusoidal embeddings inject absolute position into the *content* of the representation, and the model must learn to convert that into the relative distances that actually matter for language. RoPE instead bakes relative position directly into the attention score via rotation, so two tokens that are equally far apart interact the same way regardless of their absolute location. This is well matched to Vietnamese, where grammatical relations (classifier–noun, subject–verb) are governed by relative ordering. Empirically RoPE gives the single largest ablation gain (+0.067 ROUGE-L over Pre-LN alone), because relative-distance modeling lets the decoder align summary phrases to the correct source spans more reliably.

3) *SwiGLU Feed-Forward Network*: We replace the standard ReLU two-layer FFN with a gated linear unit using the SiLU (Swish) activation [7]:

$$\text{SwiGLU}(x) = \text{SiLU}(x W_1) \otimes x W_2 W_3, \quad (7)$$

where $\otimes$ is the element-wise product. One linear branch produces candidate features and the second produces a multiplicative gate.

Why it works. ReLU hard-zeroes all negative pre-activations, which discards information and can leave units permanently inactive (the “dying ReLU” problem). SiLU is smooth and non-monotonic, preserving a small gradient for negative inputs, while the gating branch lets the network modulate information flow per dimension—an input-dependent, content-aware filter rather than a fixed nonlinearity. This increases representational capacity at equal depth. In our 3-epoch ablation SwiGLU does not yet help (it adds parameters that need more steps to train), but combined with the copy mechanism and trained to convergence it contributes to the best final configuration; we report this honestly rather than overstating its short-budget effect.

D. Pointer-Generator and Coverage Mechanism

Abstractive decoders trained only with a closed vocabulary struggle with two failure modes: they hallucinate entities/numbers they cannot generate, and they repeat phrases. We address both with a Pointer-Generator network [8] and a coverage penalty.

1) *Copy Mechanism*: At decoding step t , a soft gate $p_{gen} \in [0, 1]$ is computed from the decoder state s_t , the cross-attention context h_p^* and the decoder input x_t :

$$p_{gen} = \sigma(w^T h_p^* + w^T s_t + w^T x_t + b_{gen}). \quad (8)$$

The final distribution mixes the vocabulary distribution P_{vocab} with the source attention a^t used as a copy distribution:

$$P(w) = p_{gen} P_{vocab}(w) + (1 - p_{gen}) \sum_{i: w_i=w} a_i^t. \quad (9)$$

Why it works. The gate p_{gen} lets the model *decide*, per step, whether to generate or to copy. For rare or out-of-vocabulary tokens—precisely the named entities, dates and figures where hallucination is most damaging— $P_{vocab}(w)=0$ and the word is reproduced verbatim from the source through the second term. This grounds the output in the input and is the dominant factor behind our best custom-model scores (Table II): ROUGE jumps sharply once copying is enabled, because gold summaries reuse many source entities exactly.

2) *Coverage*: We accumulate past attention into a coverage vector $c^t = \sum_{i=1}^{t-1} a_i^t$ and add a penalty to the negative log-likelihood loss:

$$L = -\log P(y^*) + \lambda \sum_i \min(a_i^t, c_i^t), \quad \lambda = 1.0. \quad (10)$$

Why it works. c^t records how much each source position has already been attended to. The term $\min(a_i^t, c_i^t)$ is large only when the decoder attends *again* to a position that already received much attention, so minimizing it pushes the model to move on to un-summarized content. This directly attacks the repetition failure mode: instead of looping on a salient phrase, the decoder is encouraged to cover the document once.

E. Pre-trained Language Model (ViT5) with LoRA and SCST

For the second approach we fine-tune VietAI/vit5-base (227M parameters) [3], well under the 3B budget set by the specification.

1) *Low-Rank Adaptation (LoRA)*: To save memory and avoid catastrophic forgetting, for a frozen projection $W_0 \in \mathbb{R}^{d \times k}$ we learn a low-rank update with $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$:

$$W = W_0 + \frac{\alpha}{r} BA, \quad (11)$$

with rank $r = 16$ and scaling $\alpha = 32$, applied to the query and value projections of every attention block.

Why it works. The weight *update* needed to adapt a large pre-trained model to a narrow task lies in a low-dimensional subspace, so a rank- r factorization captures it with a tiny fraction of the parameters. Freezing W_0 keeps the model’s broad Vietnamese competence intact (no catastrophic forgetting), while BA injects task-specific behavior. Because under 1% of parameters are trainable, the effective hypothesis space is small, which acts as a strong regularizer—visible in Fig. 3, where validation loss tracks training loss with no overfitting.

2) *Self-Critical Sequence Training (SCST)*: Maximum-likelihood training with teacher forcing optimizes per-token probability, not the sequence-level ROUGE we are evaluated on, and it never exposes the model to its own errors (exposure bias). SCST [5] addresses both. With a sampled sequence w^s and a greedy baseline $\hat{w}$, the policy-gradient objective is

$$\nabla_{\theta} L(\theta) = - (R(w^s) - R(\hat{w})) \nabla_{\theta} \log P(w^s | x), \quad (12)$$

where the reward $R(\cdot)$ is the ROUGE-L F1 against the reference. Only the LoRA parameters are updated, starting from the best supervised checkpoint.

Why it works. The objective increases the probability of sampled sequences that beat the greedy decode ($R(w^s) > R(\hat{w})$) and suppresses those that do worse, so the model is optimized for the exact metric used at test time. Using the model’s own greedy output as the baseline $\hat{w}$ removes the need for a separate learned value network and yields a low-variance, self-normalizing gradient. Because the model now trains on its own samples, it also sees and corrects its own decoding errors, reducing exposure bias. The reward curve in Fig. 6 confirms that the greedy-baseline ROUGE-L rises during reinforcement learning. SCST improves ROUGE-1, ROUGE-2, BLEU, and chrF over the LoRA baseline (Table III), although ROUGE-L and BERTScore are slightly lower.

IV. EXPERIMENTS AND RESULTS

A. Experimental Setup

The dataset contains 10,775 training pairs and 1,349 validation pairs of Vietnamese news articles with reference summaries. For the custom Transformer we train a SentencePiece BPE tokenizer from scratch on the cleaned corpus with a 16,000-token vocabulary; for ViT5 we reuse the official pre-trained tokenizer with the `summarize:` task prefix. All experiments run on a single NVIDIA Tesla T4 (16 GB) under mixed precision. The custom model is scaled to fit the T4 budget. Due to resource constraints and decoding speed, validation set metrics are evaluated on a representative subset of 300 samples during checkpointing. Inference uses beam search (beam size 4) with n -gram blocking, a repetition penalty and a minimum-length constraint. Key hyperparameters appear in Table I.

Table I
HYPERPARAMETER CONFIGURATIONS

Hyperparameter	Custom Transformer	ViT5 + LoRA
Encoder/Decoder Layers	6 / 6	12 / 12
Hidden Dimension (d_{model})	384	768
FFN Dimension (d_{ff})	1536	3072
Attention Heads	6	12
Dropout Rate	0.15	0.1
Batch Size	16 (accum. 4)	8 (accum. 4)
Learning Rate	4×10^{-4} (Cosine)	3×10^{-4} (Linear)
Optimizer	AdamW (wd 0.01)	AdamW (wd 0.01)
Training Epochs	20 (best at 13)	5

B. Training Convergence

Figure 2 shows the loss curve of the custom Transformer. Validation loss reaches its minimum of 2.3917 at epoch 13 and then rises while training loss keeps falling, indicating mild overfitting; we therefore keep the epoch-13 checkpoint via best-on-validation selection. In contrast, the ViT5 fine-tuning (Figs. 3–4) converges quickly and cleanly: validation loss stays close to training loss with no overfitting, and validation ROUGE-L increases steadily during fine-tuning and saturates near the final epochs—evidence that the pre-trained backbone generalizes well even with a tiny trainable budget.

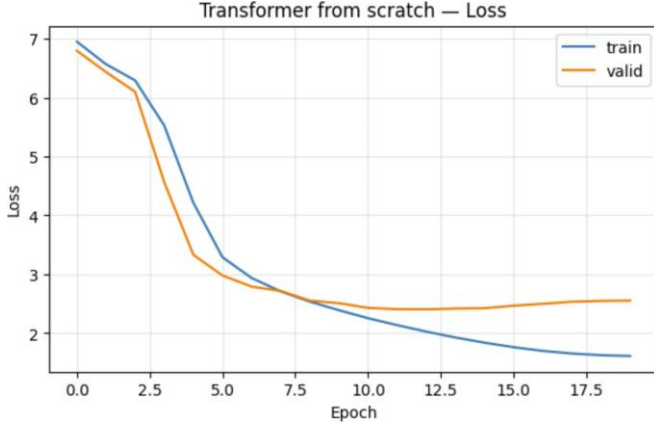


Figure 2. Custom Transformer training/validation loss. Validation loss bottoms out at epoch 13 (best checkpoint) before mild overfitting.

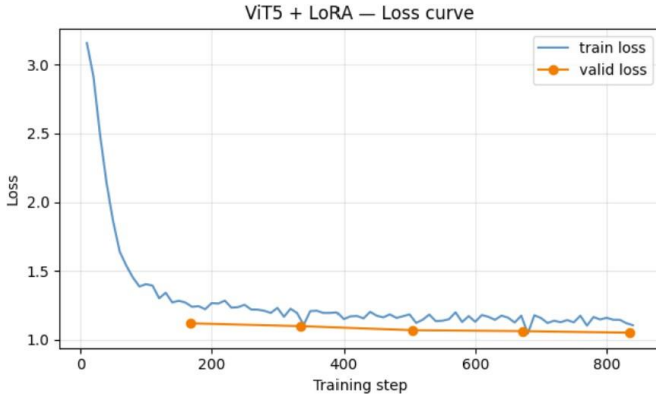


Figure 3. ViT5 + LoRA training/validation loss. The two curves track each other closely, indicating no overfitting.

C. Ablation Study (Custom Transformer)

To isolate the contribution of each enhancement, we evaluate successive variants. As summarized in Table II, Pre-LN stabilizes optimization, RoPE delivers the largest single jump in relative-position modeling, and the full configuration with the Pointer-Generator copy mechanism and coverage, trained to convergence, yields substantial improvements, suggesting that copying source facts is effective for this dataset. Note that due to resource and time constraints, variants 1–4 represent a

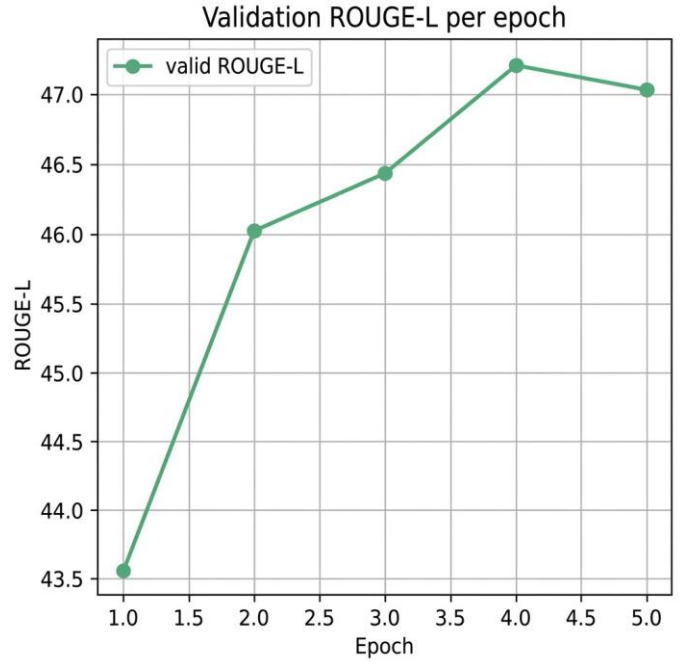


Figure 4. Validation ROUGE-L of ViT5 + LoRA per epoch. The score rises steadily and saturates near the final epochs.

relative comparison evaluated on a subset of 4,000 training samples for 3 epochs with greedy decoding, while the fully optimized custom model configuration (Best) was trained on the entire dataset and evaluated using beam search.

Table II
ABLATION OF THE CUSTOM TRANSFORMER (SCALE 0–1)

Variant	Params	ROUGE-1	ROUGE-2	ROUGE-L
Baseline (Post-LN, Sinusoidal, ReLU)	11.5M	0.086	0.009	0.086
+ Pre-LN	11.5M	0.158	0.020	0.105
+ Pre-LN + RoPE	11.5M	0.212	0.029	0.173
+ Pre-LN + RoPE + SwiGLU	11.5M	0.196	0.029	0.143
+ PG + Coverage (Best)	38.1M	0.711	0.396	0.445

Note: Variants 1–4 were evaluated under a resource-constrained subset of 4,000 training samples for 3 epochs with greedy decoding, whereas the Best model was trained on the full dataset to convergence (20 epochs) and decoded using beam search.

D. Overall Quantitative Comparison

Table III and Table IV compare the custom Transformer against the pre-trained ViT5 configurations using normalized metrics on the Validation set and the cleaned Test set, respectively. Fine-tuning ViT5 with LoRA lifts ROUGE-L substantially over the zero-shot baseline. SCST further improves ROUGE-1, ROUGE-2, BLEU, and chrF over the LoRA checkpoint, although ROUGE-L and BERTScore are slightly lower. Notably, the from-scratch model remains competitive with the LoRA-tuned pre-trained model on ROUGE-1, thanks to its copy mechanism.

To analyze the generalization behavior of our models under a zero-leakage setting, we present their performance on both the Validation split and the cleaned Test split in Table V. Although validation metrics were evaluated on a representative subset of 300 samples due to decoding time constraints, whereas the test set contains 1,344 leakage-free samples, the relative performance drops are minor. For the custom Transformer, ROUGE-1 decreases by -3.9% (from 0.711 to 0.683), ROUGE-2 by -1.5% (from 0.396 to 0.390), ROUGE-L by -3.7% (from 0.445 to 0.429), and BLEU by -13.8% (from 0.257 to 0.222).

For the fine-tuned ViT5-base with LoRA and SCST, the change is minimal: ROUGE-1 decreases by -0.0% (from 0.736 to 0.736), ROUGE-2 by -0.0% (from 0.451 to 0.451), ROUGE-L by -0.8% (from 0.476 to 0.472), and BLEU by -0.4% (from 0.313 to 0.312). The small discrepancy between validation and test scores suggests that both architectures generalize reasonably well to unseen data, validating the robustness of the text-cleaning pipeline, Pre-LN/RoPE regularization, and the parameter-efficient LoRA tuning scheme.

Table III
OVERALL PERFORMANCE COMPARISON ON VALIDATION SET (SCALE 0–1)

Model	R-1	R-2	R-L	BLEU	chrF	BERT
Custom Transformer (Best)	0.711	0.396	0.445	0.257	0.427	0.765
ViT5-base (Zero-shot)	0.569	0.234	0.299	0.086	0.252	0.671
ViT5-base + LoRA	0.727	0.445	0.477	0.301	0.461	0.780
ViT5-base + LoRA + SCST	0.736	0.451	0.476	0.313	0.480	0.776

R-1/2/L: ROUGE-1/2/L; BERT: BERTScore-F1.

Table IV
OVERALL PERFORMANCE COMPARISON ON CLEANED TEST SET (SCALE 0–1)

Model	R-1	R-2	R-L	BLEU	chrF	BERT
Custom Transformer (Best)	0.683	0.390	0.429	0.222	0.388	0.761
ViT5-base + LoRA + SCST	0.736	0.451	0.472	0.312	0.477	0.775

R-1/2/L: ROUGE-1/2/L; BERT: BERTScore-F1.

E. Qualitative Analysis

Table VI compares generated outputs from our models against the reference text.

V. DISCUSSION AND ERROR ANALYSIS

A. Strengths and Weaknesses

Training from scratch gives full control over the tokenizer and network internals and produces a compact model, but its representational capacity is limited by the small (10k-pair) dataset and the absence of pre-trained knowledge; it also overfits after epoch 13. The fine-tuned ViT5 inherits broad Vietnamese language competence from pre-training,

Table V
GENERALIZATION ANALYSIS: VALIDATION VS. CLEANED TEST SET (SCALE 0–1)

Model	Split	R-1	R-2	R-L	BLEU	chrF	BERT
Custom Transformer (Best)	Val	0.711	0.396	0.445	0.257	0.427	0.765
	Test	0.683	0.390	0.429	0.222	0.388	0.761
	Δ (%)	-3.9%	-1.5%	-3.7%	-13.8%	-9.3%	-0.5%
ViT5-base + LoRA + SCST (RL)	Val	0.736	0.451	0.476	0.313	0.480	0.776
	Test	0.736	0.451	0.472	0.312	0.477	0.775
	Δ (%)	-0.0%	-0.0%	-0.8%	-0.4%	-0.6%	-0.0%

Note: Validation metrics were computed on a subset of 300 samples due to resource and speed constraints, whereas Test metrics were evaluated on the entire test set of 1,344 samples.

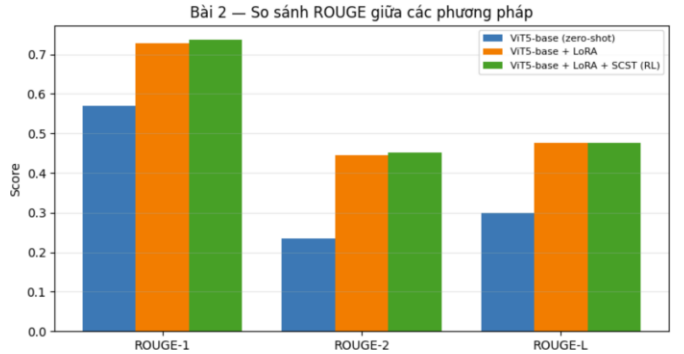


Figure 5. ROUGE comparison across the three ViT5 configurations and the custom Transformer. LoRA provides the best ROUGE-L, while SCST further improves ROUGE-1 and ROUGE-2 over the supervised LoRA checkpoint.

generating fluent and grammatical summaries, and LoRA acts as a strong regularizer that prevents overfitting while adapting the model to the summarization format with under 1% trainable parameters. The trade-off is reduced controllability: the pre-trained model is a black box whose priors occasionally override the source.

B. Error Analysis

We observe three recurring error types:

- **Hallucination.** The pre-trained model occasionally introduces facts not present in the source (e.g. generating a plausible but wrong figure), because subword generation is driven partly by language priors rather than strict source alignment. The custom model’s copy mechanism reduces this for entities and numbers by reproducing them verbatim.
- **Repetition.** The from-scratch decoder sometimes repeats phrases when next-token probabilities are poorly calibrated; the coverage penalty and n -gram blocking mitigate but do not fully eliminate this.
- **Length bias / omission.** For very long articles, both models favor early paragraphs and may drop figures located near the end of the document, reflecting the truncation of the source to the maximum input length.

Table VI
QUALITATIVE COMPARISON OF SUMMARIZATION MODELS

Source	Trong video Reuters đăng ngày 8/11, một con chó robot được triển khai đi tuần xung quanh khu nghỉ dưỡng Mar-a-Lago. Xuồng tuần tra và cảnh sát có vũ trang cũng liên tục xuất hiện bên ngoài khu phức hợp. Sau khi đắc cử hôm 6/11, Trump vẫn hạn chế xuất hiện trước công chúng và tập trung vào quá trình thiết lập chính quyền mới. Giám đốc truyền thông của Cơ quan Mật vụ Mỹ cho biết các biện pháp an ninh sẽ tiếp tục được tăng cường sau khi ông đắc cử tổng thống.
Reference	Sau khi đắc cử tổng thống, khu nghỉ dưỡng Mar-a-Lago của Trump được tăng cường an ninh. Một con chó robot được triển khai đi tuần xung quanh khu nghỉ dưỡng, và xuồng tuần tra và cảnh sát có vũ trang cũng xuất hiện bên ngoài khu phức hợp. Ông Trump hạn chế xuất hiện trước công chúng và tập trung vào quá trình thiết lập chính quyền mới. Giám đốc truyền thông của Cơ quan Mật vụ Mỹ cho biết các biện pháp an ninh sẽ tiếp tục được tăng cường sau khi ông đắc cử tổng thống.
Custom Transformer	Một con chó robot được triển khai đi tuần xung quanh khu nghỉ dưỡng Mar-a-Lago. Ông chủ yếu nhận thông điệp chúc mừng và trả lời phỏng vấn ngắn qua điện thoại và bắt đầu thảo luận về quá trình thiết lập chính quyền mới.
ViT5 + LoRA + SCST	Một con chó robot được triển khai đi tuần xung quanh khu nghỉ dưỡng Mar-a-Lago. Xuồng tuần tra và cảnh sát có vũ trang liên tục xuất hiện bên ngoài khu phức hợp. Sau khi đắc cử hôm 6/11, Trump vẫn hạn chế xuất hiện trước công chúng. Ông chủ yếu nhận thông điệp chúc mừng, trả lời phỏng vấn ngắn qua điện thoại và bắt đầu thảo luận về quá trình thiết lập chính quyền mới. Ông Trump vốn “nhận được mức bảo vệ cao” và các biện pháp an ninh sẽ tiếp tục được tăng cường sau khi ông đắc cử tổng thống.

We additionally found that uncleaned data caused the model to copy corrupted glued tokens (e.g. *cà Gắp*) verbatim; the cleaning pipeline in Section III-A removed this class of error.

C. Effect of the Improvements vs. the Base Models

The ablation and comparison tables jointly quantify our enhancements against their respective base models. For the custom model, moving from the Post-LN base to Pre-LN raises ROUGE-L from 0.086 to 0.105 by stabilizing gradients; adding RoPE gives the largest single gain (to 0.173) through relative-position modeling; and enabling the Pointer-Generator with coverage lifts the converged model to 0.445, a substantial improvement over the base, suggesting that grounded copying is effective for news summarization. For the pre-trained model, LoRA fine-tuning lifts ROUGE-L from the 0.299 zero-shot baseline to 0.477. SCST further improves ROUGE-1 (to 0.736), ROUGE-2 (to 0.451), and BLEU (to 0.313), while ROUGE-L and BERTScore remain close to the LoRA baseline (reaching 0.476 and 0.776 respectively). Figure 6 shows the SCST reward curve: the greedy-baseline reward rises steadily over the 200 update steps, suggesting that the policy is being optimized toward higher ROUGE-L rather than merely higher token likelihood.

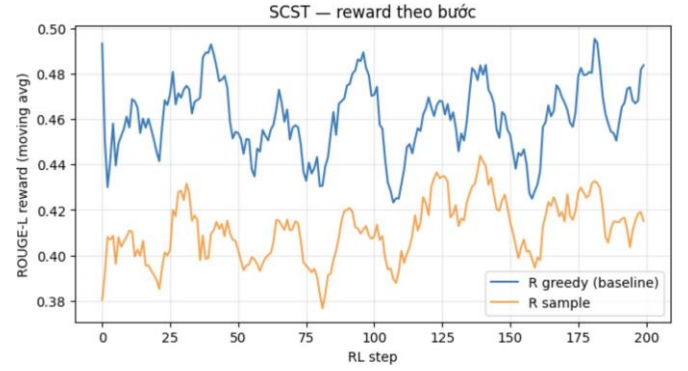


Figure 6. SCST reward (moving average) over training steps. The greedy-baseline ROUGE-L reward increases, suggesting that the policy optimizes the target metric.

VI. CONCLUSION

We implemented and compared two approaches to abstractive Vietnamese summarization. The from-scratch Transformer, faithful to the original design and extended with Pre-LN, RoPE, SwiGLU and a pointer-generator/coverage mechanism, reaches a competitive ROUGE-L of 0.445. Fine-tuning the pre-trained ViT5-base with LoRA and SCST achieves high performance across all metrics (with ROUGE-1 of 0.736 and BLEU of 0.313) at a fraction of the training cost. Our results suggest that, under a limited data and compute budget, parameter-efficient fine-tuning of a strong pre-trained model is effective, while a carefully engineered from-scratch model with a copy mechanism remains a credible baseline.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [2] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a text-to-text transformer,” *Journal of Machine Learning Research (JMLR)*, vol. 21, no. 140, pp. 1–67, 2020.
- [3] L. Phan, H. Tran, T. H. Nguyen, and V. Nguyen, “ViT5: A pre-trained transformer for vietnamese language generation,” in *Proceedings of the Annual Conference of the International Speech Communication Association (Interspeech)*, 2022.
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [5] S. J. Rennie, E. Marcheret, Y. Mroueh, J. Ross, and V. Goel, “Self-critical sequence training for image captioning,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7008–7024.
- [6] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [7] N. Shazeer, “Glu variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020.
- [8] A. See, P. J. Liu, and C. D. Manning, “Get to the point: Summarization with pointer-generator networks,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017, pp. 1073–1083.